

Effect of ill-conditioned Jacobians in training of Normalizing Flows

Correlation between loss function and condition number of
Jacobian

Project Course 4MA453 supervised by Jonas Nordqvist

Jan Kristof Köglburger

February 24, 2026

Contents

1	Introduction	2
1.1	Organization of the report	2
1.2	Our contribution	2
2	Preliminaries	3
2.1	Notation	3
2.2	Kullback-Leibler divergence	3
3	Normalizing Flows	4
3.1	Constructing a Normalizing Flow	4
3.2	Linear Flow	5
3.3	Expressive Power of Normalizing Flows	6
3.4	RealNVP	7
3.5	Fitting a Normalizing Flow	8
3.5.1	Minimizing Kullback-Leibler divergence	8
4	Determinant and condition number	10
4.1	Preliminaries	10
4.2	Jacobian of a RealNVP coupling layer	11
5	Experiments	16
5.1	Code	16
5.2	Metrics	16
5.3	Figures	16
5.3.1	Moons	17
5.3.2	Circles	18
5.3.3	Blobs	20
5.4	Analysis of results in dimension 2	21
5.5	Embedding two-dimensional data in higher dimensions	22
5.6	Experiments in dimension 4	23
5.6.1	Moons	23
5.6.2	Circles	25
5.6.3	Blobs	26
5.6.4	Analysis of results	28
6	Conclusion	29
A	Python Code	31

1 Introduction

On one-hand this report aims to give a comprehensive overview of the concept of a Normalizing Flow. This includes justification, underlying mathematical properties and examples of a Normalizing Flow architecture.

On the other hand we investigate if the loss function and the condition number of the Jacobian matrix used in the training of the Normalizing Flow are linearly correlated. We do this empirically in Python but also derive theoretical results that support our experimental results.

1.1 Organization of the report

In Section 3 we motivate the concept of a Normalizing Flow and gradually introduce all the necessary mathematical tools for its construction. We present the simple example of a Linear Flow and introduce RealNVP, the Normalizing Flow architecture that we will focus on in Sections 4 and 5. Besides that, in Section 3 we will also present a proof of the theoretical expressive power of Normalizing Flows and show how to fit a Normalizing Flow by minimizing the Kullback-Leibler divergence.

Section 4 contains theoretical results about the relationship between the determinant and the condition number of the Jacobian matrix in the case of RealNVP.

In Section 5 we explain the setup of our experiments and present the figures we created along with an analysis of the results.

1.2 Our contribution

All the propositions in Section 4 are formulated and proven by the author of this report. The simple example of a Linear Flow in Section 3.2 was created by the author.

The basic code in Python for a Normalizing Flow with RealNVP coupling layers was provided by the supervisor. The author of this report supplemented it with the computation of the metrics and the embedding of the two-dimensional data in higher dimensions.

2 Preliminaries

2.1 Notation

For a random variable X we denote its probability density function (PDF) by p_X . We also use p_X to denote the probability distribution of X .

2.2 Kullback-Leibler divergence

Let p_X and p_Y be probability density functions in $\mathbb{R}^n$.

The Kullback-Leibler (KL) divergence [KL51] of p_X from p_Y is defined as

$$\text{KL}(p_X \parallel p_Y) = \int_{\mathbb{R}^n} p_X(x) \log \left(\frac{p_X(x)}{p_Y(x)} \right) dx. \quad (1)$$

The defining properties of a divergence hold. The KL divergence is always non-negative and it is zero if and only if $p_X = p_Y$. It is however, not symmetric (there are p_X and p_Y such that $\text{KL}(p_X \parallel p_Y) \neq \text{KL}(p_Y \parallel p_X)$). Moreover, the triangle inequality does not hold in general. Therefore the KL divergence is not a metric.

The differential entropy of p_X is defined as

$$\text{H}(p_X) = - \int_{\mathbb{R}^n} p_X(x) \log(p_X(x)) dx.$$

Adding the KL divergence and the differential entropy results in the cross-entropy

$$\begin{aligned} \text{H}(p_X, p_Y) &:= \text{KL}(p_X \parallel p_Y) + \text{H}(p_X) \\ &= \int_{\mathbb{R}^n} p_X(x) \log \left(\frac{p_X(x)}{p_Y(x)} \right) dx - \int_{\mathbb{R}^n} p_X(x) \log(p_X(x)) dx \\ &= \int_{\mathbb{R}^n} (p_X(x)(\log(p_X(x)) - \log(p_Y(x))) - p_X(x) \log(p_X(x))) dx \\ &= - \int_{\mathbb{R}^n} p_X(x) \log(p_Y(x)) dx. \end{aligned}$$

3 Normalizing Flows

The probability distribution which generates the data we encounter in machine learning is usually unknown and quite complex. The basic idea behind Normalizing Flows is to transform such an unknown complex probability distribution into a simple (usually standard Gaussian) probability distribution. Since this transformation is chosen to be bijective, one can then evaluate the density function of the data distribution. One can also use the bijective transformation to sample from the complex data distribution.

3.1 Constructing a Normalizing Flow

Let n be the dimension of the data that we want to work with. We choose a so-called base distribution in $\mathbb{R}^n$ which has a simple and familiar probability density function (PDF). We denote this PDF at the point u by $p_U(u)$. Usually the n -dimensional standard normal distribution is chosen as the base distribution.

We denote the unknown PDF of the data distribution at x by $p_X(x)$. By data distribution we mean the unknown probability distribution followed by the data points that we encounter.

Our goal is to find a bijective function $D : \mathbb{R}^n \rightarrow \mathbb{R}^n$, $D(u) = x$ that maps any vector u sampled from the base distribution to a vector x sampled from the data distribution. Moreover, we want to express $p_X(x)$ in terms of $p_U(u)$ and D .

Hence we further require D to be differentiable and have a differentiable inverse, i.e. D has to be a diffeomorphism. Under these conditions it holds that

$$p_X(x) = p_U(u) |\det J_D(u)|^{-1}, \quad (2)$$

where

$$J_D(u) = \begin{pmatrix} \frac{\partial D_1}{\partial u_1} & \cdots & \frac{\partial D_1}{\partial u_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial D_n}{\partial u_1} & \cdots & \frac{\partial D_n}{\partial u_n} \end{pmatrix} \quad (3)$$

is the Jacobian matrix of D at the point u (cf. [Bog07]).

Of course we need to ask the question for which base and data distributions we can find such a diffeomorphism D . In Section 3.3 we will show that a non-zero PDF and the differentiability of certain conditional probabilities are sufficient to guarantee the existence of D .

In practice Normalizing Flows are usually implemented as neural networks and composed of several layers. This is possible since the composition of diffeomorphisms on $\mathbb{R}^n$ is again a diffeomorphism on $\mathbb{R}^n$: Let $T : \mathbb{R}^n \rightarrow \mathbb{R}^n$, $T(x) = z$ be another diffeomorphism which maps any vector x sampled from the distribution p_X to a vector z which is sampled from p_Z . Then $V = T \circ D$ is a

diffeomorphism with $V(u) = z$ and

$$\begin{aligned} p_Z(z) &= p_U(u) |\det J_V(u)|^{-1} \\ &= p_U(u) |\det J_{T \circ D}(u)|^{-1} \\ &= p_U(u) |\det J_T(D(u)) \cdot \det J_D(u)|^{-1} \\ &= p_U(u) |\det J_T(D(u))|^{-1} |\det J_D(u)|^{-1}. \end{aligned}$$

Composing a number of layers lets one use a relatively simple structure in each layer but still model complex distributions. A simple structure, especially a Jacobian determinant that is easy to compute, is crucial for an efficient implementation of Normalizing Flows.

3.2 Linear Flow

Let us now look at a simple example of a Normalizing Flow. Consider the n -dimensional standard normal distribution as the base distribution, i.e.

$$p_U(u) = \frac{\exp(-\frac{1}{2}u^T u)}{(2\pi)^{n/2}}.$$

For an invertible $n \times n$ matrix A and a n -dimensional vector b we define

$$D : \mathbb{R}^n \rightarrow \mathbb{R}^n, D(u) = Au + b =: x.$$

This linear function is a diffeomorphism since A is invertible. Moreover, $J_D(u) = A$ and thus

$$p_X(x) = p_U(u) |\det J_D(u)|^{-1} = \frac{\exp(-\frac{1}{2}u^T u)}{(2\pi)^{n/2}} \cdot \frac{1}{|\det A|}$$

From $x = Au + b$ it follows that $u = A^{-1}(x - b)$. Using standard rules for the transpose and inverse of matrices and setting $\Sigma = AA^T$ we get

$$\begin{aligned} u^T u &= (A^{-1}(x - b))^T A^{-1}(x - b) \\ &= (x - b)^T (A^{-1})^T A^{-1}(x - b) \\ &= (x - b)^T (AA^T)^{-1}(x - b) \\ &= (x - b)^T (\Sigma)^{-1}(x - b). \end{aligned}$$

Since

$$\det \Sigma = \det AA^T = \det A \cdot \det A^T = (\det A)^2$$

it follows that

$$|\det A| = (\det \Sigma)^{\frac{1}{2}}$$

and thus

$$p_X(x) = \frac{\exp(-\frac{1}{2}(x - b)^T (\Sigma)^{-1}(x - b))}{(2\pi)^{n/2} (\det \Sigma)^{\frac{1}{2}}}. \quad (4)$$

Equation (4) is exactly the density of a n -dimensional normal distribution with mean b and covariance matrix Σ . So choosing a linear function as our diffeomorphism D is only sufficient if we want to model a normal distribution. For more complex data distributions we need to use non-linear diffeomorphisms.

3.3 Expressive Power of Normalizing Flows

Following [Pap+21], we will now prove that theoretically any data distribution p_X that follows two regularity conditions can be modeled from any base distribution p_U following the same conditions by a Normalizing Flow.

Let p_X be a distribution for which $p_X(x) > 0$ for all $x \in \mathbb{R}^n$ and the conditional probabilities

$$P(X_i \leq x_i | X_1 \leq x_1, X_2 \leq x_2, \dots, X_{i-1} \leq x_{i-1}) \quad (5)$$

are differentiable with respect to $(x_1, x_2, \dots, x_i)$. Here the random variable X_i is the i -th entry of the random variable X , i.e. $X = (X_1, X_2, \dots, X_n)^T$. For the partial vector before the i -th entry we introduce the notation

$$x_{<i} = (x_1, x_2, \dots, x_{i-1})$$

and we denote equation (5) by $P(X_i \leq x_i | x_{<i})$.

Our PDF can be decomposed into a product of conditional density functions as

$$p_X(x) = \prod_{i=1}^n p_X(x_i | x_{<i}).$$

Since $p_X(x) > 0$ for all $x \in \mathbb{R}^n$, it follows that also $p_X(x_i | x_{<i}) > 0$ for all i .

We define $T : \mathbb{R}^n \rightarrow (0, 1)^n$, $x \rightarrow z$ element-wise by

$$z_i = T_i(x) = T_i(x_1, \dots, x_i) = \int_{-\infty}^{x_i} p_X(y_i | x_{<i}) dy_i = P(X_i \leq x_i | x_{<i}).$$

Each T_i is differentiable and therefore T is differentiable. The derivative

$$\frac{\partial T_i}{\partial x_i} = p_X(x_i | x_{<i})$$

is positive everywhere and therefore $T_i(\cdot, x_{<i}) : \mathbb{R} \rightarrow (0, 1)$ is invertible. The $z_i = T_i(x_1, \dots, x_i)$ do not depend on any x_j where $j > i$. Hence we can invert T element by element writing

$$x_i = T_i(\cdot, x_{<i})^{-1}(z_i).$$

Since $\frac{\partial F_i}{\partial x_j} = 0$ for $j > i$, the Jacobian of T is lower triangular and we can compute its determinant by

$$\det J_T(x) = \prod_{i=1}^n \frac{\partial T_i}{\partial x_i} = \prod_{i=1}^n p_X(x_i, x_{<i}) = p_X(x)$$

which is positive everywhere. The inverse function, whose existence we have already proven, is therefore also differentiable and T is a diffeomorphism.

Analogous to equation (2) we get

$$p_Z(z) = p_X(x) |\det J_T(x)|^{-1} = p_X(x) p_X(x)^{-1} = 1$$

which means that p_Z is a uniform distribution on $(0, 1)^n$.

Using the inverse transformation T^{-1} we get $x = T^{-1}(z)$ and

$$p_X(x) = p_Z(z) |\det J_{T^{-1}}(z)|^{-1} \quad (6)$$

which shows that p_X can be modeled by a Normalizing Flow from a uniform distribution on $(0, 1)^n$.

We now extend the proof to any base distribution p_U that is following the regularity conditions.

Let p_U be a distribution for which $p_U(u) > 0$ for all $u \in \mathbb{R}^n$ and the conditional probabilities

$$P(U_i \leq u_i | U_1 \leq u_1, U_2 \leq u_2, \dots, U_{i-1} \leq u_{i-1}) \quad (7)$$

are differentiable with respect to $(u_1, u_2, \dots, u_i)$.

We define $D : \mathbb{R}^n \rightarrow (0, 1)^n$, $u \rightarrow z$ element-wise by

$$z_i = D_i(u) = D_i(u_1, \dots, u_i) = \int_{-\infty}^{u_i} p_U(y_i | u_{<i}) dy_i = P(U_i \leq u_i | u_{<i}).$$

By the exact same reasoning as above, D is a diffeomorphism and

$$p_Z(z) = p_U(u) |\det J_D(x)|^{-1} = p_U(u) p_U(u)^{-1} = 1.$$

Now $F = T^{-1} \circ D$, $F(u) = x$ is a diffeomorphism and thus

$$p_X(x) = p_U(u) |\det J_F(u)|^{-1}$$

which shows that p_X can be modeled from p_U by a Normalizing Flow.

3.4 RealNVP

Let $x \in \mathbb{R}^n$, $n > d \in \mathbb{N}$ and let $\odot$ denote element-wise multiplication. The notation $x_{1:d}$ means the vector that contains the first d components of the vector x , while $x_{d+1:n}$ means the vector that one gets when truncating the first d components of x . Moreover let s and t be functions from $\mathbb{R}^d \rightarrow \mathbb{R}^{n-d}$. These functions are usually implemented by a neural network. By $\exp(s(x_{1:d}))$ we mean that the exponential function is applied to every element of the vector $s(x_{1:d})$.

We define the RealNVP coupling layer [DSB17] by

$$\begin{aligned} f(x)_{1:d} &= x_{1:d} \\ f(x)_{d+1:n} &= x_{d+1:n} \odot \exp(s(x_{1:d})) + t(x_{1:d}). \end{aligned}$$

The Jacobian of f is given by the block matrix

$$J_f(x) = \begin{pmatrix} \mathbb{I}_d & 0 \\ M & \text{diag}(\exp(s(x_{1:d}))) \end{pmatrix}, \quad (8)$$

where

$$M = \begin{pmatrix} \frac{\partial f(x)_{d+1}}{\partial x_1} & \dots & \frac{\partial f(x)_{d+1}}{\partial x_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial f(x)_n}{\partial x_1} & \dots & \frac{\partial f(x)_n}{\partial x_d} \end{pmatrix}.$$

Since $J_f(x)$ is a triangular matrix, its determinant is given by

$$\det(J_f(x)) = \prod_{j=1}^{n-d} \exp(s(x_{1:d})_j) = \exp\left(\sum_{j=1}^{n-d} s(x_{1:d})_j\right) \quad (9)$$

and

$$\log |\det(J_f(x))| = \sum_{j=1}^{n-d} s(x_{1:d})_j. \quad (10)$$

3.5 Fitting a Normalizing Flow

In the following section, we will show how to fit a model that is based on a Normalizing Flow to a data distribution p_X .

The parametrized distribution is given by

$$\hat{p}_{X_\theta}(x) = p_U(u) |\det J_{D_\theta}(u)|^{-1} \quad (11)$$

where p_U is the base distribution and θ are the parameters of the diffeomorphism D . Note that we could also let p_U depend on some parameters.

Our goal is to bring $\hat{p}_{X_\theta}$ as close as possible to p_X . This requires a measure for the distance between two distributions. A popular choice is the Kullback-Leibler divergence defined in equation (1).

3.5.1 Minimizing Kullback-Leibler divergence

The KL divergence of p_X from $\hat{p}_{X_\theta}$ is given by

$$\text{KL}(p_X \parallel \hat{p}_{X_\theta}) = \int_{\mathbb{R}^n} p_X(x) \log\left(\frac{p_X(x)}{\hat{p}_{X_\theta}(x)}\right) dx.$$

Using a standard rule for the logarithm and rewriting the integrals as expected values leads to

$$\begin{aligned} \text{KL}(p_X \parallel \hat{p}_{X_\theta}) &= \int_{\mathbb{R}^n} p_X(x) \log(p_X(x)) dx - \int_{\mathbb{R}^n} p_X(x) \log(\hat{p}_{X_\theta}(x)) dx \\ &= \mathbb{E}_{p_X}[\log(p_X(x))] - \mathbb{E}_{p_X}[\log(\hat{p}_{X_\theta}(x))]. \end{aligned}$$

We set

$$C := \mathbb{E}_{p_X}[\log(p_X(x))]$$

since this term does not depend on θ and we can therefore treat it as a constant without caring about its exact value. Using equation (11) and rewriting the Jacobian determinant in terms of the inverse function we get

$$\begin{aligned}
\text{KL}(p_X || \hat{p}_{X_\theta}) &= -\mathbb{E}_{p_X} [\log(\hat{p}_{X_\theta}(x))] + C \\
&= -\mathbb{E}_{p_X} [\log(p_U(u) | \det J_{D_\theta}(u)|^{-1})] + C \\
&= -\mathbb{E}_{p_X} [\log(p_U(u)) + \log(|\det J_{D_\theta}(u)|^{-1})] + C \\
&= -\mathbb{E}_{p_X} [\log(p_U(D_\theta^{-1}(x)) + \log(|\det J_{D_\theta^{-1}}(x)|))] + C.
\end{aligned}$$

Let $\{x_1, \dots, x_m\}$ be a set of independent samples drawn from the data distribution p_X . Then the KL divergence above can be estimated by

$$-\frac{1}{m} \sum_{i=1}^m (\log(p_U(D_\theta^{-1}(x_i)) + \log(|\det J_{D_\theta^{-1}}(x_i)|)) + C \quad (12)$$

since the law of large numbers ensures that this converges almost surely to $-\mathbb{E}_{p_X} [\log(p_U(D_\theta^{-1}(x)) + \log(|\det J_{D_\theta^{-1}}(x)|))] + C$ when $m \rightarrow \infty$.

Hence we can fit the model to the samples $\{x_1, \dots, x_m\}$ by minimizing Expression (12) with respect to the parameters θ .

4 Determinant and condition number

The following section contains theoretical results regarding condition numbers and determinants with a focus on the Jacobian matrices of RealNVP coupling layers.

4.1 Preliminaries

The singular value decomposition (SVD) of a matrix $A \in M^{n \times n}(\mathbb{R})$ is the factorization

$$A = U \Sigma V^T,$$

where U and V are orthogonal matrices and Σ is a diagonal matrix that contains the singular values $\sigma_i = \Sigma_{ii}$ of A . The singular values are the positive square roots of the eigenvalues of $A^T A$.

The condition number (with respect to the Euclidean norm) is defined by

$$\kappa(A) := \frac{\max_i(\sigma_i)}{\min_i(\sigma_i)}. \quad (13)$$

The matrices U and V are orthogonal, thus we have $\det(U) = \pm 1$ and $\det(V^T) = \det(V) = \pm 1$.

Since Σ is a diagonal matrix, it holds that

$$\det(\Sigma) = \sigma_1 \cdot \sigma_2 \cdots \sigma_n.$$

The singular values are always non-negative, hence

$$|\det(A)| = \det(\Sigma) = \sigma_1 \cdot \sigma_2 \cdots \sigma_n.$$

It can be shown (see [Hig21]) that for a triangular matrix $A = (a_{ij})$ we have

$$\kappa(A) \geq \frac{\max_i |a_{ii}|}{\min_i |a_{ii}|}. \quad (14)$$

Lemma 1. *For $p \in \mathbb{R}$ it holds that*

$$p - \sqrt{p^2 - 1} \xrightarrow{p \rightarrow \infty} 0.$$

Proof. First note that for all $1 \leq p \in \mathbb{R}$ we have $\sqrt{p^2 - 1} \in \mathbb{R}$. Since $p + \sqrt{p^2 - 1} \neq 0$ for all $p \in \mathbb{R}$, we can rewrite

$$p - \sqrt{p^2 - 1} = \left(p - \sqrt{p^2 - 1} \right) \cdot \frac{p + \sqrt{p^2 - 1}}{p + \sqrt{p^2 - 1}} = \frac{p^2 - (p^2 - 1)}{p + \sqrt{p^2 - 1}} = \frac{1}{p + \sqrt{p^2 - 1}}.$$

Clearly

$$p + \sqrt{p^2 - 1} \xrightarrow{p \rightarrow \infty} \infty$$

which implies the desired result

$$p - \sqrt{p^2 - 1} \xrightarrow{p \rightarrow \infty} 0. \quad \square$$

4.2 Jacobian of a RealNVP coupling layer

We present some results regarding the relation between the determinant and the condition number of the Jacobian matrix of a RealNVP coupling layer.

Proposition 1. *Let the Jacobian determinant of a RealNVP coupling layer be very close to 0. More precisely, let*

$$\det(J_f(x)) = \prod_{j=1}^{n-d} \exp(s(x_{1:d})_j) = \exp\left(\sum_{j=1}^{n-d} s(x_{1:d})_j\right) =: \epsilon,$$

where $\sqrt[n-d]{\epsilon} \ll 1$. Then

$$\kappa(J_f(x)) \geq \frac{1}{\sqrt[n-d]{\epsilon}} \gg 1.$$

Proof. We first need to show that there exists $k \in \{1, \dots, n-d\}$ such that

$$\exp(s(x_{1:d})_k) \leq \sqrt[n-d]{\epsilon}. \quad (15)$$

Suppose $\exp(s(x_{1:d})_j) > \sqrt[n-d]{\epsilon}$ for all $j \in \{1, \dots, n-d\}$. Then

$$\det(J_f(x)) = \prod_{j=1}^{n-d} \exp(s(x_{1:d})_j) > (\sqrt[n-d]{\epsilon})^{n-d} = \epsilon,$$

which is a contradiction to $\det(J_f(x)) = \epsilon$. Hence there exists an index k which satisfies inequality (15).

We know that the Jacobian of the RealNVP coupling layer is given by the block matrix

$$J_f(x) = \begin{pmatrix} \mathbb{I}_d & 0 \\ M & \text{diag}(\exp(s(x_{1:d}))) \end{pmatrix}.$$

It follows that $\min_i |J_f(x)_{ii}| \leq \exp(s(x_{1:d})_k) \leq \sqrt[n-d]{\epsilon}$ and $\max_i |J_f(x)_{ii}| \geq 1$.

This together with the bound for the condition number (inequality (14)) implies that

$$\kappa(J_f(x)) \geq \frac{\max_i |J_f(x)_{ii}|}{\min_i |J_f(x)_{ii}|} \geq \frac{1}{\sqrt[n-d]{\epsilon}}. \quad (16)$$

The assumption $\sqrt[n-d]{\epsilon} \ll 1$ obviously leads to

$$\frac{1}{\sqrt[n-d]{\epsilon}} \gg 1$$

which completes the proof. $\square$

Loosely speaking, Proposition 1 shows that an "almost" singular Jacobian of a RealNVP coupling layer necessarily has a high condition number. The opposite, however, is not true, as shown in the following proposition.

Proposition 2. *There exists a RealNVP coupling layer f and an infinite sequence of points $z_1, z_2, \dots$ such that*

$$\kappa(J_f(z_i)) \xrightarrow{i \rightarrow \infty} \infty,$$

while

$$\det(J_f(z_i)) = 1$$

for all $i \in \mathbb{N}$.

Proof. Let f be the RealNVP coupling layer in 2 dimensions where s and t are both given by the identity function on $\mathbb{R}$. This means that $s(x_1) = t(x_1) = x_1$ and $s'(x_1) = t'(x_1) = 1$. Therefore the Jacobian of f at the point $x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$ is given by

$$J_f(x) = \begin{pmatrix} 1 & 0 \\ x_2 \cdot \exp(x_1) + 1 & \exp(x_1) \end{pmatrix}.$$

Consider the sequence of points

$$z_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, z_2 = \begin{pmatrix} 0 \\ 2 \end{pmatrix}, \dots, z_n = \begin{pmatrix} 0 \\ n \end{pmatrix}, \dots$$

For all $i \in \mathbb{N}$ we have

$$\det(J_f(z_i)) = 1 \cdot \exp(0) = 1.$$

To get the singular values of $J_f(z_i)$ we have to calculate the eigenvalues of $J_f^T(z_i) \cdot J_f(z_i)$ and take their positive square roots. We have

$$\begin{aligned} J_f^T(z_i) \cdot J_f(z_i) &= \begin{pmatrix} 1 & i+1 \\ 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 \\ i+1 & 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 + (i+1)^2 & i+1 \\ i+1 & 1 \end{pmatrix} \end{aligned}$$

and

$$\begin{aligned} \det(J_f^T(z_i) \cdot J_f(z_i) - \lambda I) &= \det \begin{pmatrix} 1 + (i+1)^2 - \lambda & i+1 \\ i+1 & 1 - \lambda \end{pmatrix} \\ &= \lambda^2 - \lambda(2 + (i+1)^2) + 1. \end{aligned}$$

The equation

$$\lambda^2 - \lambda(2 + (i+1)^2) + 1 = 0$$

has the solutions

$$\lambda_1 = \frac{2 + (i+1)^2}{2} + \sqrt{\left(\frac{2 + (i+1)^2}{2}\right)^2 - 1}$$

and

$$\lambda_2 = \frac{2 + (i+1)^2}{2} - \sqrt{\left(\frac{2 + (i+1)^2}{2}\right)^2 - 1}.$$

By Lemma 1 we have $\lambda_1 \xrightarrow{i \rightarrow \infty} \infty$ and $\lambda_2 \xrightarrow{i \rightarrow \infty} 0$. Hence we get the singular values $\sigma_1 = \sqrt{\lambda_1} \xrightarrow{i \rightarrow \infty} \infty$ and $\sigma_2 = \sqrt{\lambda_2} \xrightarrow{i \rightarrow \infty} 0$ and with equation (13) the desired result

$$\kappa(J_f(z_i)) = \frac{\sqrt{\lambda_1}}{\sqrt{\lambda_2}} \xrightarrow{i \rightarrow \infty} \infty. \quad \square$$

Proposition 2 shows that there exist RealNVP coupling layers whose Jacobians have an arbitrarily large condition number but still a determinant of 1 which is far from being singular.

Proposition 3 improves the result of Proposition 2 by constructing such a RealNVP coupling layer in any dimension n (and for any choice of d). Note that in the proof of Proposition 3 the functions s and t are different from those in the proof of Proposition 2.

Proposition 3. *In every dimension n there exists a RealNVP coupling layer f (with d being an arbitrary integer satisfying $1 \leq d \leq n-1$) and an infinite sequence of points $z_1, z_2, \dots$ such that*

$$\kappa(J_f(z_i)) \xrightarrow{i \rightarrow \infty} \infty,$$

while

$$\det(J_f(z_i)) = 1$$

for all $i \in \mathbb{N}$.

Proof. Let n and d be arbitrary integers satisfying $n \geq 2$ and $1 \leq d \leq n-1$.

We first define $s : \mathbb{R}^d \rightarrow \mathbb{R}^{n-d}$ to be the zero function, i.e. $s(x) = 0$ for all $x \in \mathbb{R}^d$. We further define $t : \mathbb{R}^d \rightarrow \mathbb{R}^{n-d}$ by

$$t \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{pmatrix} = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ \frac{x_1^2}{2} \end{pmatrix}.$$

The induced RealNVP coupling layer f has a sparse Jacobian matrix. None of the components $f(x)_{d+1}, \dots, f(x)_{n-1}$ depends on any of the variables $x_1, \dots, x_d$. The last component $f(x)_n$ depends only on x_1 and has partial derivative

$$\frac{\partial f(x)_n}{\partial x_1} = \frac{\partial \frac{x_1^2}{2}}{\partial x_1} = x_1.$$

Hence the Jacobian matrix of f (equation (8)) contains only one off-diagonal non-zero entry and is given by

$$J_f(x) = \begin{pmatrix} 1 & & \\ & \ddots & \\ x_1 & & 1 \end{pmatrix}$$

Obviously $\det(J_f(x)) = 1$ for all $x \in \mathbb{R}^n$.

To get the singular values of $J_f(x)$ we calculate the eigenvalues of $J_f(x) \cdot J_f^T(x)$ and take their positive square roots. We have

$$J_f(x) \cdot J_f^T(x) = \begin{pmatrix} 1 & & \\ & \ddots & \\ x_1 & & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & & x_1 \\ & \ddots & \\ & & 1 \end{pmatrix} = \begin{pmatrix} 1 & & x_1 \\ & \ddots & \\ x_1 & & 1 \\ & & & x_1^2 + 1 \end{pmatrix}$$

Expanding the determinant along the first column and one of the resulting sub-determinants along its last column yields

$$\begin{aligned} \det(J_f(x) \cdot J_f^T(x) - \lambda I) &= \det \begin{pmatrix} 1 - \lambda & & & x_1 \\ & \ddots & & \\ & & 1 - \lambda & \\ x_1 & & & x_1^2 + 1 - \lambda \end{pmatrix} \\ &= (1 - \lambda) \cdot \det \begin{pmatrix} 1 - \lambda & & & \\ & \ddots & & \\ & & 1 - \lambda & \\ & & & x_1^2 + 1 - \lambda \end{pmatrix} \\ &\quad + (-1)^{n+1} \cdot x_1 \cdot \det \begin{pmatrix} 0 & & & x_1 \\ 1 - \lambda & \ddots & & \\ & \ddots & \ddots & \\ & & 1 - \lambda & 0 \end{pmatrix} \\ &= (1 - \lambda)(1 - \lambda)^{n-2}(x_1^2 + 1 - \lambda) \\ &\quad + (-1)^{n+1} \cdot x_1 \cdot (-1)^n \cdot x_1 \cdot \det \begin{pmatrix} 1 - \lambda & & & \\ & \ddots & & \\ & & 1 - \lambda & \end{pmatrix} \\ &= (1 - \lambda)(1 - \lambda)^{n-2}(x_1^2 + 1 - \lambda) - x_1^2 \cdot (1 - \lambda)^{n-2} \\ &= (1 - \lambda)^{n-2}((1 - \lambda)(x_1^2 + 1 - \lambda) - x_1^2) \\ &= (1 - \lambda)^{n-2}(\lambda^2 - (2 + x_1^2) \cdot \lambda + 1). \end{aligned}$$

The equation

$$(1 - \lambda)^{n-2}(\lambda^2 - (2 + x_1^2) \cdot \lambda + 1) = 0$$

has the solutions $\lambda_1 = 1$ (with multiplicativity $n - 2$) and

$$\lambda_{2,3} = \frac{2 + x_1^2}{2} \pm \sqrt{\left(\frac{2 + x_1^2}{2}\right)^2 - 1}.$$

We consider the sequence of points

$$z_1 = \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, z_2 = \begin{pmatrix} 2 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \dots, z_i = \begin{pmatrix} i \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \dots$$

The squared singular values of $J_f(z_i)$ are given by $\lambda_1^{(i)} = 1$,

$$\lambda_2^{(i)} = \frac{2 + i^2}{2} + \sqrt{\left(\frac{2 + i^2}{2}\right)^2 - 1}$$

and

$$\lambda_3^{(i)} = \frac{2 + i^2}{2} - \sqrt{\left(\frac{2 + i^2}{2}\right)^2 - 1}.$$

By Lemma 1 we have $\lambda_2^{(i)} \xrightarrow{i \rightarrow \infty} \infty$ and $\lambda_3^{(i)} \xrightarrow{i \rightarrow \infty} 0$.

The singular value $\sigma_2^{(i)} = \sqrt{\lambda_2^{(i)}}$ is greater than $\sigma_1^{(i)} = 1$ while $\sigma_3^{(i)} = \sqrt{\lambda_3^{(i)}}$ is smaller than $\sigma_1^{(i)} = 1$. Hence with equation (13) we get the desired result

$$\kappa(J_f(z_i)) = \frac{\max_k(\sigma_k^{(i)})}{\min_k(\sigma_k^{(i)})} = \frac{\sigma_2^{(i)}}{\sigma_3^{(i)}} = \frac{\sqrt{\lambda_2^{(i)}}}{\sqrt{\lambda_3^{(i)}}} \xrightarrow{i \rightarrow \infty} \infty. \quad \square$$

5 Experiments

5.1 Code

We implemented a Normalizing Flow with RealNVP coupling layers in Python. The basic code structure was provided by Jonas Nordqvist. It was then gradually adapted by the author to compute certain metrics. The code used for all the results that are provided in this report can be found in Appendix A.

We use the PyTorch library for the implementation of the neural network and scikit-learn to generate one of three different toy datasets (`make_moons`, `make_circles` or `make_blobs`) whose distribution we try to model by a Normalizing Flow. The functions from scikit-learn create two-dimensional data points. First we only conducted experiments in two dimensions. Later on we allowed any input dimension greater than one by embedding the two dimensional data points in higher dimensions (see Section 5.5).

The modifiable parameters are the number of data points that are created, the noise, the width of the neural network, the depth of the neural network (i.e. the number of layers), the learning rate for the optimizer, the batch size and the number of epochs (the number of times each data point is used for training).

The functions s and t in the RealNVP coupling layers (cf. Section 3.4) are also implemented by neural networks. The base distribution is a multivariate standard normal.

5.2 Metrics

Obviously we need to compute a loss function in order to train our Normalizing Flow. For any batch we use the above estimation of the KL divergence (expression (12)). In our code `log_prob_x` is a tensor containing the terms of the sum in expression (12).

We also determine the full Jacobian matrix at any data point for every RealNVP coupling layer. Using the singular value decomposition of the Jacobian, we compute its condition number as the largest singular value divided by the smallest singular value (equation (13)). Moreover, we calculate the lower bound for the condition number given by inequality (14).

We take the arithmetic mean of all the condition numbers of all layers over the batch. Likewise we compute the maximum value of all the condition numbers over the batch. Regarding the lower bound for the condition number, we also take the arithmetic mean over the batch.

Furthermore, for the first two and the last two layers of the Normalizing Flow, we compute the Pearson correlation coefficient between `log_prob_x` and the condition number of the Jacobian matrix in the respective layer.

5.3 Figures

To visualize the power of the Normalizing Flow, we create a plot containing three subplots. First, the target distribution, i.e. the data points generated by the

functions from scikit-learn. Second, random samples from the base distribution that are carried over to the target distribution via the Normalizing Flow. Third, the data points carried over to the base distribution via the Normalizing Flow.

Regarding the metrics, we display graphs of the loss, the mean condition number, the maximum condition number and the mean condition number bound. Moreover, we create graphs of the Pearson correlation coefficient for the first two and the last two layers. In these graphs, we visualize the arithmetic mean of all the correlation coefficients with a red line.

5.3.1 Moons

We present the figures for the `make_moons` dataset with dimension 2, 200 data points, a noise of 0.05, a neural network of width 32 and depth 4, a learning rate of 0.001, a batch size of 32 and 301 epochs.

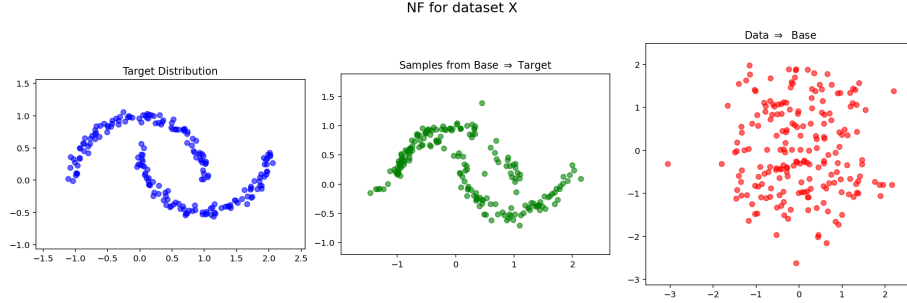


Figure 1: Plots for the `make_moons` dataset. To the left the samples from the target distribution, in the center samples from the base distribution (a standard normal) carried over to the target distribution and to the right the samples from the target distribution carried over to the base distribution.

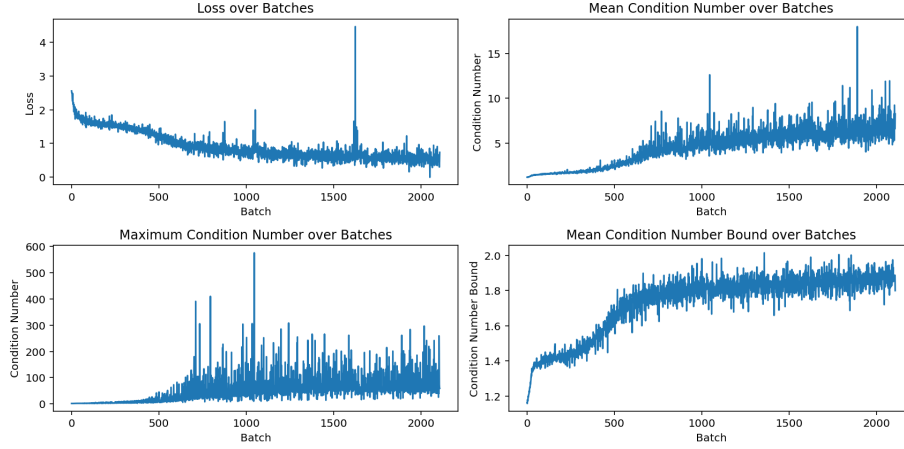


Figure 2: Graphs of the loss per batch, the mean and the maximum condition number of the Jacobian matrix per batch and the mean condition number bound per batch for the `make_moons` dataset.

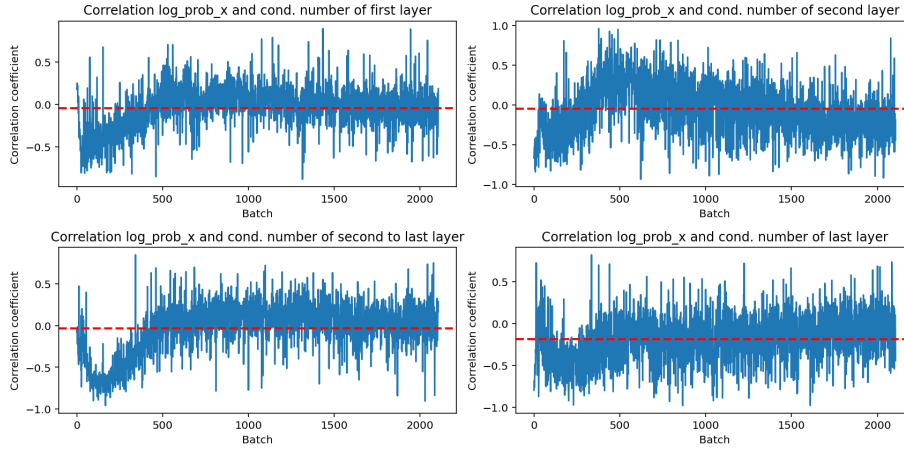


Figure 3: Graphs of the Pearson correlation coefficients between `log_prob_x` and the condition number of the Jacobian matrix per batch for the first two and the last two layers of the Normalizing Flow for the `make_moons` dataset. The red lines indicate the arithmetic mean.

5.3.2 Circles

We present the figures for the `make_circles` dataset with dimension 2, 200 data points, a noise of 0.05, a neural network of width 32 and depth 4, a learning rate of 0.001, a batch size of 32 and 301 epochs.

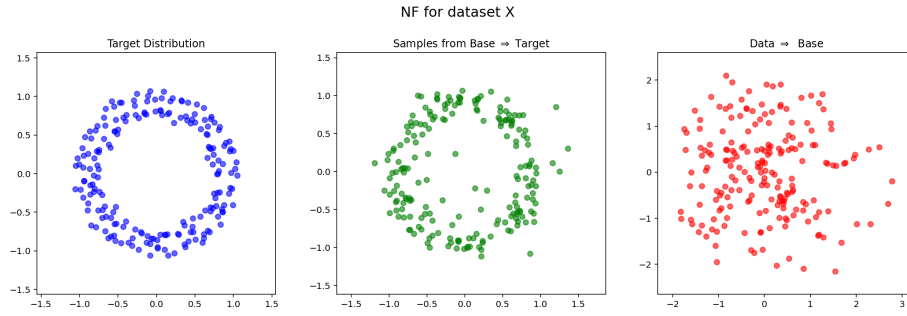


Figure 4: Plots for the `make_circles` dataset.

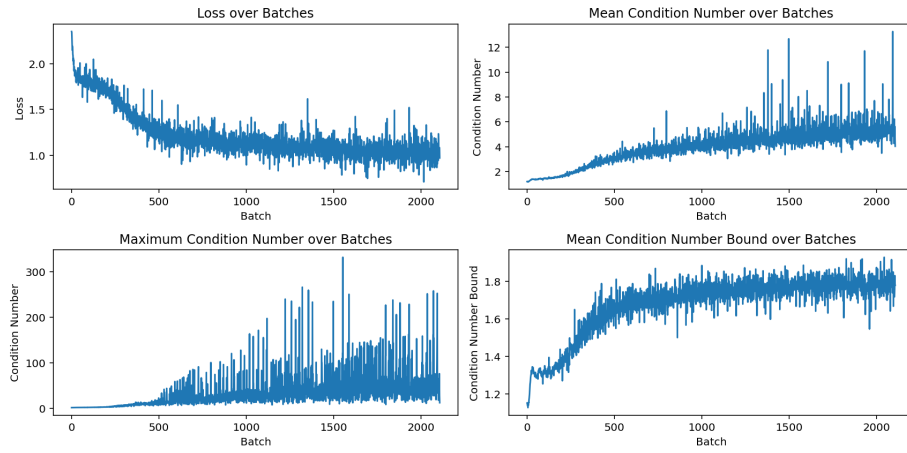


Figure 5: Graphs of the loss per batch, the mean and the maximum condition number of the Jacobian matrix per batch and the mean condition number bound per batch for the `make_circles` dataset.

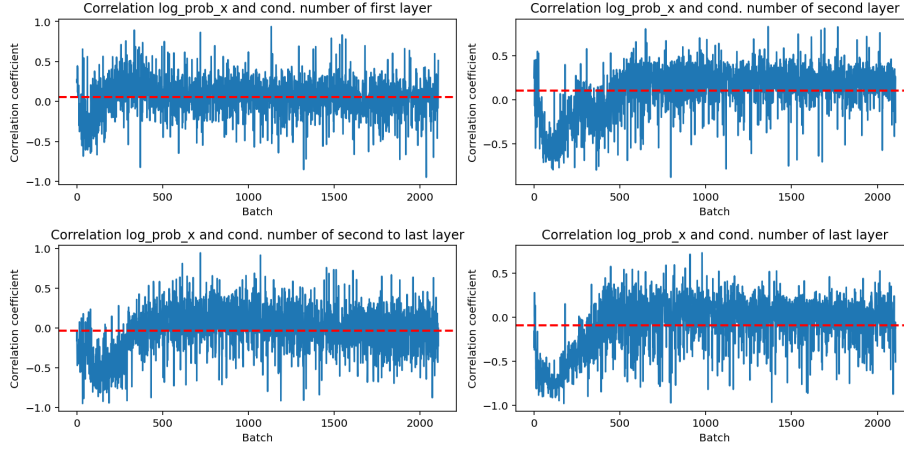


Figure 6: Graphs of the Pearson correlation coefficients between `log_prob_x` and the condition number of the Jacobian matrix per batch for the first two and the last two layers of the Normalizing Flow for the `make_circles` dataset. The red lines indicate the arithmetic mean.

5.3.3 Blobs

We present the figures for the `make_blobs` dataset with dimension 2, 200 data points, a noise of 0.05, a neural network of width 32 and depth 4, a learning rate of 0.001, a batch size of 32 and 301 epochs.

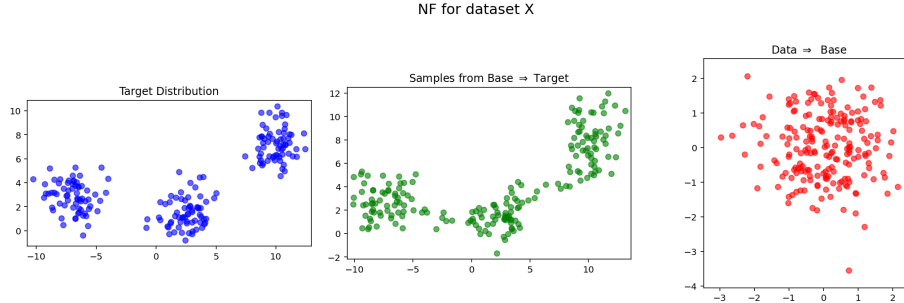


Figure 7: Plots for the `make_blobs` dataset.

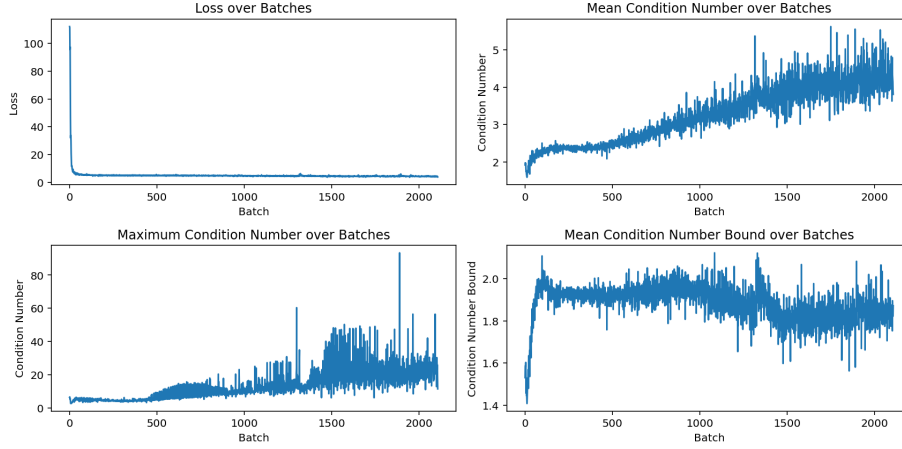


Figure 8: Graphs of the loss per batch, the mean and the maximum condition number of the Jacobian matrix per batch and the mean condition number bound per batch for the `make_blobs` dataset.

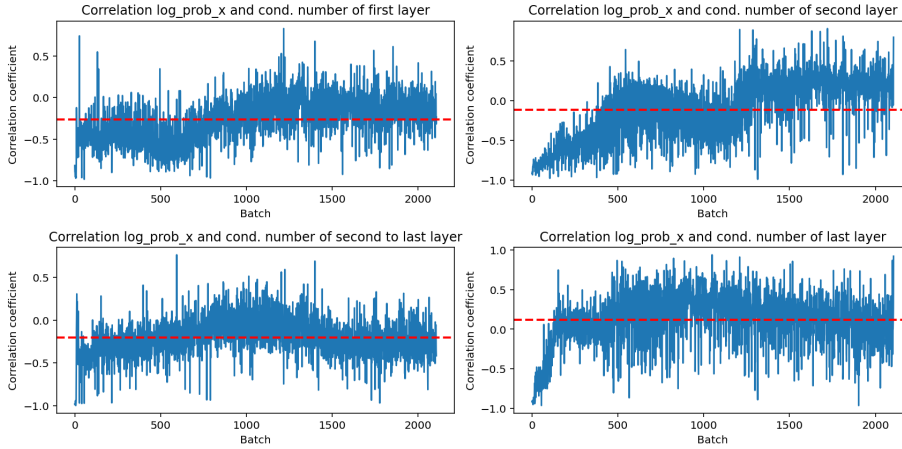


Figure 9: Graphs of the Pearson correlation coefficients between `log_prob_x` and the condition number of the Jacobian matrix per batch for the first two and the last two layers of the Normalizing Flow for the `make_blobs` dataset. The red lines indicate the arithmetic mean.

5.4 Analysis of results in dimension 2

The results are similar for all three datasets.

Visual inspection of Figures 1, 4 and 7 shows that the trained Normalizing Flow is able to model the target distribution. In all three cases, the random

samples from the base distribution carried over to the target distribution via the Normalizing Flow (the plot in the center) produce a pattern which is quite similar to the pattern of the data points (the plot on the left side). Moreover, the data points carried over to the base distribution (the plot on the right side) look how we expect it from a standard normal distribution in two dimensions.

The curves of the loss in Figures 2, 5 and 8 decrease on average but are rather flat at the end of our training. This indicates that we can not significantly improve our model by increasing the number of epochs. The loss in Figure 2 has one significant spike to a value of approximately 4.46 in batch number 1623. This seems to be an exceptional case but generally speaking, the loss fluctuates in all our examples (as well as the condition numbers and correlation coefficients). The mean condition number of the Jacobian is increasing over time. The same applies to the mean condition number bound in Figures 2 and 5. This lower bound on average rarely exceeds 2 and is significantly smaller than the mean condition number itself. Hence we do not use the lower bound instead of the condition number itself, which we originally considered to speed up the computations.

Most importantly, however, Figures 3, 6 and 9 do not indicate a linear correlation between `log_prob_x` and the condition number of the Jacobian matrix. For all three datasets and all four layers the correlation coefficients are on average close to zero.

We want to conduct more experiments to see if we can find a linear correlation when the intrinsic dimension of the data is lower than the dimension in which the Normalizing Flow is trained.

5.5 Embedding two-dimensional data in higher dimensions

To construct a case where the intrinsic dimension of the data is lower than the dimension in which the Normalizing Flow is trained, we create a function that embeds our two-dimensional datasets in any dimension $n > 2$ in a non-trivial way. Non-trivial means that we do not just map the two-dimensional data to the subspace spanned by two vectors of the standard base of $\mathbb{R}^n$ via the identity function.

Instead we first create two vectors $u_1, u_2 \in \mathbb{R}^n$ that are orthonormal. Then we define the embedding

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}^n, f(x_1, x_2) = x_1 \cdot u_1 + x_2 \cdot u_2,$$

where $\cdot$ denotes scalar multiplication. The implementation in Python is done in the following steps:

- We create two vectors v_1 and v_2 of length n where each coordinate is drawn from a uniform distribution on the interval $[-1, 1]$ (using the function `numpy.random.uniform()`).
- We perform the Gram-Schmidt orthonormalization process, i.e. we first normalize v_1 to get u_1 , then compute the vector $v_2 - \langle v_2, u_1 \rangle \cdot u_1$ and

finally normalize this vector to get u_2 . Now u_1 and u_2 are two orthonormal vectors in $\mathbb{R}^n$.

- We create an "embedding matrix" that has the rows u_1 and u_2 . Our data points are stored in the rows of another matrix. We multiply the matrix containing the data points by the embedding matrix. Now we have data points in $\mathbb{R}^2$.

The function f is mapping $\mathbb{R}^2$ to the two-dimensional subspace that is spanned by u_1 and u_2 in $\mathbb{R}^n$. To map this subspace back to $\mathbb{R}^2$ we can simply transpose the matrix that we used to map data points from $\mathbb{R}^2$ to $\mathbb{R}^n$. If u_1 and u_2 are written as row vectors and the embedding matrix as

$$M = \begin{pmatrix} u_1 \\ u_2 \end{pmatrix},$$

then

$$M \cdot M^T = \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} \cdot \begin{pmatrix} u_1^T & u_2^T \end{pmatrix} = \begin{pmatrix} \langle u_1, u_1 \rangle & \langle u_1, u_2 \rangle \\ \langle u_2, u_1 \rangle & \langle u_2, u_2 \rangle \end{pmatrix},$$

where $\langle \cdot, \cdot \rangle$ denotes the dot product. Since u_1 and u_2 are orthonormal vectors, we have

$$\begin{pmatrix} \langle u_1, u_1 \rangle & \langle u_1, u_2 \rangle \\ \langle u_2, u_1 \rangle & \langle u_2, u_2 \rangle \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I_2.$$

After training the Normalizing Flow in Python, we can send samples from the base distribution through the flow to get samples from the target distribution. However, these samples will be data points in $\mathbb{R}^n$ which we expect to lie in (or close to) the subspace. To display these samples in a two-dimensional plot, we multiply the matrix which contains the data points as rows with the matrix M^T .

5.6 Experiments in dimension 4

Through an exhaustive search we found parameters for each of the three datasets such that in dimension 4 the Normalizing Flow is able to model the target distribution.

5.6.1 Moons

For the `make_moons` dataset we used the following parameters: 2048 data points, a noise of 0.1, a neural network of width 128 and depth 64, a learning rate of 0.0005, a batch size of 16 and 51 epochs.

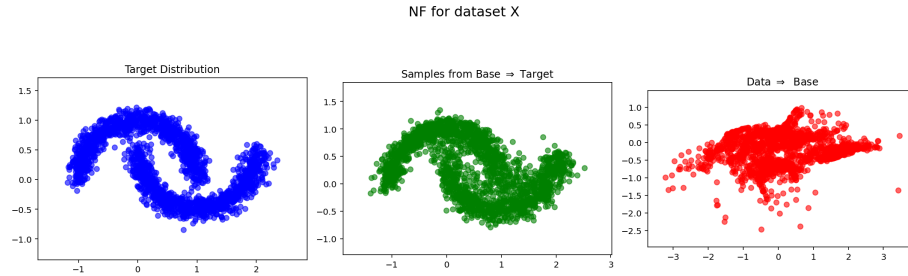


Figure 10: Plots for the `make_moons` dataset trained in dimension 4.

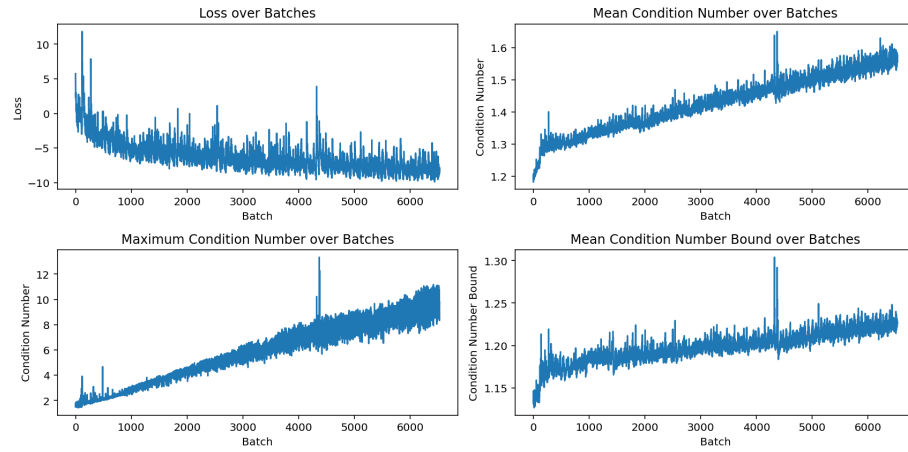


Figure 11: Graphs of the loss per batch, the mean and the maximum condition number of the Jacobian matrix per batch and the mean condition number bound per batch for the `make_moons` dataset trained in dimension 4.

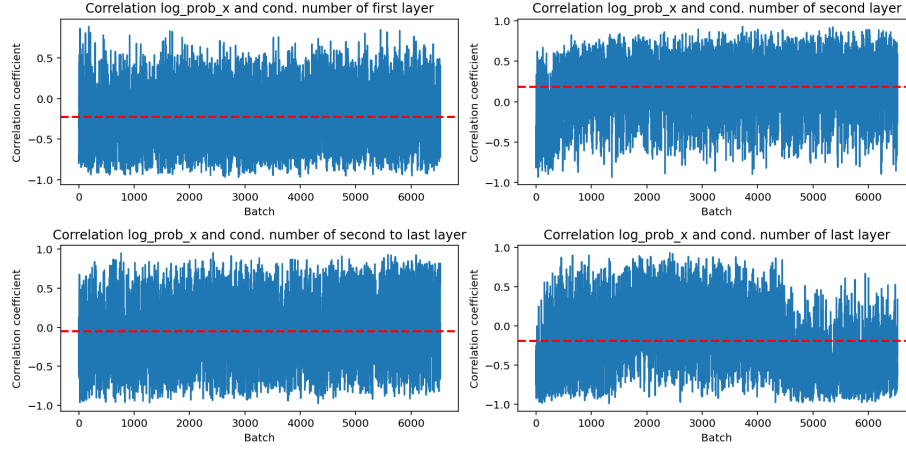


Figure 12: Graphs of the Pearson correlation coefficients between `log_prob_x` and the condition number of the Jacobian matrix per batch for the first two and the last two layers of the Normalizing Flow for the `make_moons` dataset trained in dimension 4. The red lines indicate the arithmetic mean.

5.6.2 Circles

For the `make_circles` dataset we used the following parameters: 256 data points, a noise of 0.05, a neural network of width 32 and depth 32, a learning rate of 0.001, a batch size of 16 and 101 epochs.

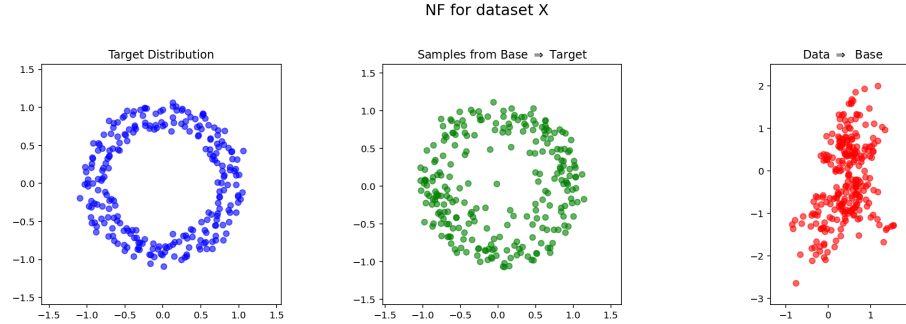


Figure 13: Plots for the `make_circles` dataset trained in dimension 4.

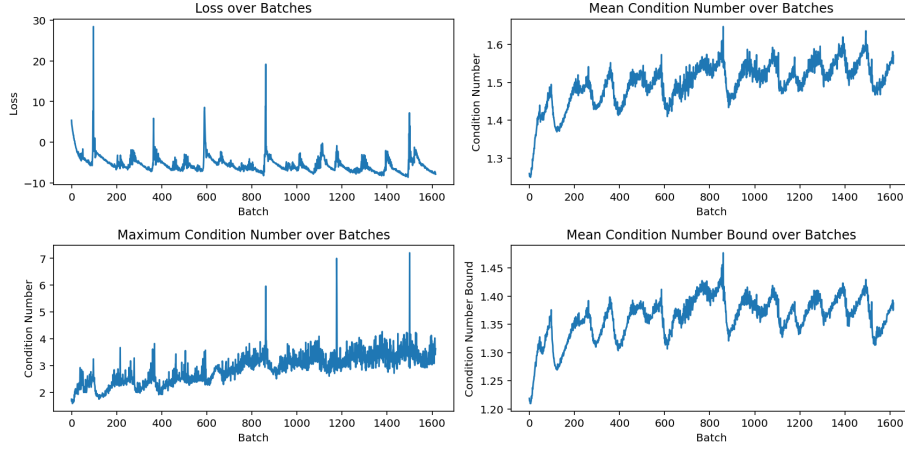


Figure 14: Graphs of the loss per batch, the mean and the maximum condition number of the Jacobian matrix per batch and the mean condition number bound per batch for the `make_circles` dataset trained in dimension 4.

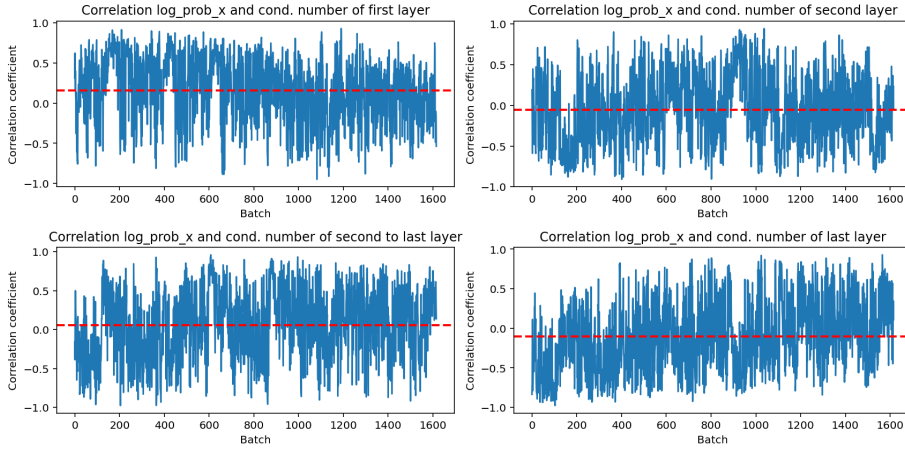


Figure 15: Graphs of the Pearson correlation coefficients between `log_prob_x` and the condition number of the Jacobian matrix per batch for the first two and the last two layers of the Normalizing Flow for the `make_circles` dataset trained in dimension 4. The red lines indicate the arithmetic mean.

5.6.3 Blobs

For the `make_blobs` dataset we used the following parameters: 1024 data points, a noise of 0.5, a neural network of width 64 and depth 16, a learning rate of 0.005, a batch size of 64 and 201 epochs.

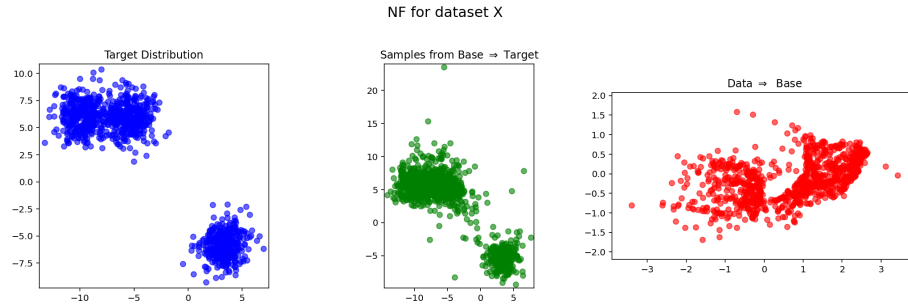


Figure 16: Plots for the `make_blobs` dataset trained in dimension 4.

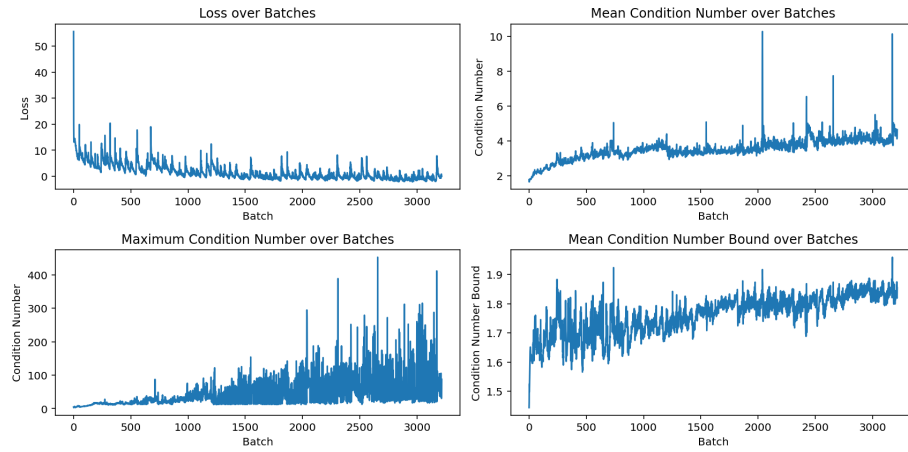


Figure 17: Graphs of the loss per batch, the mean and the maximum condition number of the Jacobian matrix per batch and the mean condition number bound per batch for the `make_blobs` dataset trained in dimension 4.

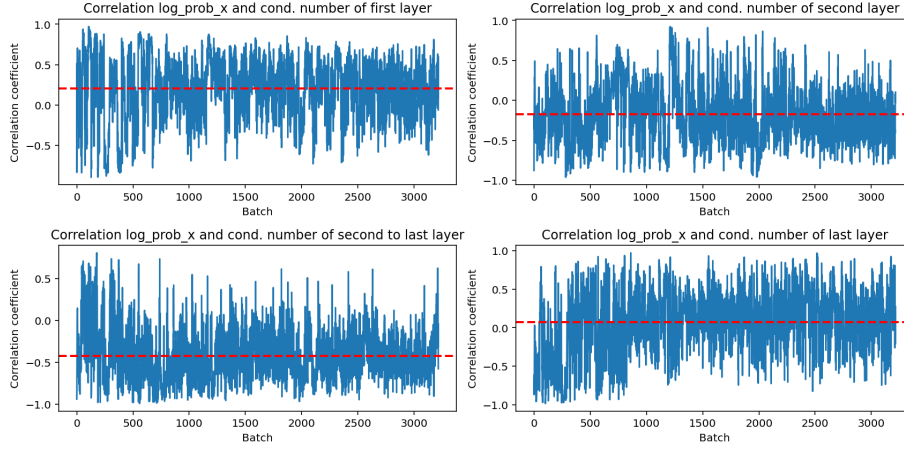


Figure 18: Graphs of the Pearson correlation coefficients between $\log_prob_x$ and the condition number of the Jacobian matrix per batch for the first two and the last two layers of the Normalizing Flow for the `make_blobs` dataset trained in dimension 4. The red lines indicate the arithmetic mean.

5.6.4 Analysis of results

The results in dimension 4 are very similar to those in dimension 2.

Visual inspection of Figures 10, 13 and 16 shows that the trained Normalizing Flow is able to model the target distribution with the parameters we chose. Note that in Figure 16 the plots on the left side and in the center have different scales. This is because the plot in the center, which shows random samples from the base distribution carried over to the target distribution via the Normalizing Flow, contains some outliers. However, the vast majority of the samples still lies in blobs which strongly resemble the pattern of the data points in the plot on the left side.

Generally speaking, in Figures 11, 14 and 17 the loss decreases over time while the various metrics of the condition number increase over time (although there are a lot of ups and downs).

Most importantly, the graphs of the Pearson correlation coefficients in Figures 12, 15 and 18 do not indicate a linear correlation between $\log_prob_x$ and the condition number of the Jacobian matrix. The correlation coefficients are on average close to zero.

6 Conclusion

The experiments in Section 5 do not show a significant linear correlation between the terms of the loss function (`log_prob_x`) and the condition numbers of the Jacobian matrices.

Clearly the loss function is influenced by the determinants of the Jacobian matrices due to how it is defined in expression (12).

A possible reason why we could not detect a linear correlation between `log_prob_x` and the condition numbers is provided by the theoretical results in Section 4. They show that in the case of RealNVP coupling layers the Jacobians can have arbitrarily large condition numbers while being far from singular. If the determinant is exactly 1 like in Proposition 3, then the logarithm of the determinant is 0. Hence in this case the high condition number of the Jacobian does not affect the loss function (expression (12)) at all.

References

- [Bog07] Vladimir I. Bogachev. *Measure Theory*. Berlin, Heidelberg: Springer, 2007. ISBN: 978-3-540-34513-8.
- [CT06] Thomas M. Cover and Joy A. Thomas. *Elements of information theory*. eng. 2. ed. Hoboken, N.J: Wiley, 2006. ISBN: 0471241954.
- [DSB17] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. *Density estimation using Real NVP*. 2017. arXiv: 1605.08803 [cs.LG]. URL: <https://arxiv.org/abs/1605.08803>.
- [GBC16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [Hig21] Nick Higham. *Bounds for the Norm of the Inverse of a Triangular Matrix*. 2021. URL: <https://nhigham.com/2021/03/30/bounds-for-the-norm-of-the-inverse-of-a-triangular-matrix/> (visited on 10/13/2025).
- [KL51] S. Kullback and R. A. Leibler. “On Information and Sufficiency”. In: *The Annals of Mathematical Statistics* 22.1 (1951), pp. 79–86. DOI: 10.1214/aoms/1177729694. URL: <https://doi.org/10.1214/aoms/1177729694>.
- [Pap+21] George Papamakarios et al. “Normalizing Flows for Probabilistic Modeling and Inference”. In: *Journal of Machine Learning Research* 22.57 (2021), pp. 1–64. URL: <http://jmlr.org/papers/v22/19-1028.html>.
- [QSS07] Alfio Quarteroni, Riccardo Sacco, and Fausto Saleri. *Numerical Mathematics*. eng. 2nd ed. 2007. Texts in Applied Mathematics, 37. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007. ISBN: 3-540-49809-5.

A Python Code

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.distributions import MultivariateNormal
from sklearn.datasets import make_moons, make_circles, make_blobs
import matplotlib.pyplot as plt
import numpy as np
import copy

# Configurations
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
N_SAMPLES = 200
NOISE = 0.05
INPUT_DIM = 2
INTRINSIC_DIM = 2 # intrinsic dimension of input data
HIDDEN_DIM = 32 # width of the neural network
N_LAYERS = 4 # depth of the neural network
LR = 1e-3 # learning rate for the optimizer
BATCH_SIZE = 32 # was 32
N_EPOCHS = 300 # number of times each data points is used for
                training

class CouplingLayer(nn.Module):
    """
    An implementation of a RealNVP-style coupling layer.
    The input is split in half. One half is used to predict the
    scale and shift
    parameters for the other half.
    """
    def __init__(self, input_dim, hidden_dim, mask):
        super().__init__()
        self.mask = mask

        self.s_t_net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, input_dim * 2)
        )

    def forward(self, x):
        """ The forward pass for training (data -> latent space). """
        # Create a masked version of the input to feed into the
        network
        x_masked = x * self.mask

        # Compute scale and shift parameters from the masked input
        s_t = self.s_t_net(x_masked)
        s, t = s_t.chunk(2, dim=-1)
```

```

s = s * (1 - self.mask)
t = t * (1 - self.mask)

# Use tanh to stabilize training and prevent explosion
s = torch.tanh(s)

# Apply the transformation:  $y = x * \exp(s) + t$ 
y = x * torch.exp(s) + t

# The log-determinant of the Jacobian is the sum of the log
-scaling factors.
log_det_J = s.sum(dim=-1)

return y, log_det_J

def inverse(self, y):
    """ The inverse pass for sampling (latent (base) space ->
data). """
    y_masked = y * self.mask

    s_t = self.s_t_net(y_masked)
    s, t = s_t.chunk(2, dim=-1)

    s = s * (1 - self.mask)
    t = t * (1 - self.mask)

    s = torch.tanh(s)
    x = (y - t) * torch.exp(-s)

    return x

# Placeholder for the implementation of the computations of the
Jacobian (suggestion is to use AutoGrad)
def compute_jacobian(self, x):
    x = x.clone().requires_grad_(True) # Ensure we retain
graph for gradient calculation
    #jacobian = torch.zeros(x.size(0), x.size(1), x.size(1)) #
dummy to replace later

    # Compute Jacobian per sample
    jacobian = []
    for i in range(x.size(0)):
        x_i = x[i].detach().clone().requires_grad_(True) #
individual input
        f_i = lambda x: self.forward(x)[0]
        J_i = torch.autograd.functional.jacobian(f_i, x_i) #
shape: (output_dim, input_dim)
        jacobian.append(J_i)

    # Stack into tensor of shape (batchsize, output_dim,
input_dim)
    jacobian = torch.stack(jacobian)

    #print("Jacobian shape:", jacobian.shape) # torch.Size([
batchsize, 2, 2])
    return jacobian

```

```

class NF(nn.Module):
    def __init__(self, input_dim, hidden_dim, n_layers):
        super().__init__()

        self.layers = nn.ModuleList()

        # Alternating mask for the coupling layer
        for i in range(n_layers):
            mask = torch.zeros(input_dim)
            mask[i % 2::2] = 1 # Alternate wich half is masked
            self.layers.append(CouplingLayer(input_dim, hidden_dim,
            mask.to(DEVICE)))

    def forward(self, x):
        """ Maps data to the base distribution. """
        total_log_det_J = 0
        for layer in self.layers:
            x, log_det_J = layer(x)
            total_log_det_J += log_det_J
        return x, total_log_det_J

    def inverse(self, z):
        """ Maps the base distribution to data. """
        for layer in reversed(self.layers):
            z = layer.inverse(z)
        return z

    def compute_jacobians(self, x):
        """Computes a list of Jacobian matrices for each layer."""
        jacobians = []
        for layer in self.layers:
            jacobians.append(layer.compute_jacobian(x))
            x, _ = layer(x)
        return jacobians

base_distribution = MultivariateNormal(
    loc=torch.zeros(INPUT_DIM).to(DEVICE),
    covariance_matrix=torch.eye(INPUT_DIM).to(DEVICE)
)

# Various types of datasets
X, _ = make_moons(n_samples=N_SAMPLES, noise=NOISE)
#X, _ = make_circles(n_samples=N_SAMPLES, noise=NOISE)
#X, _ = make_blobs(n_samples = N_SAMPLES, n_features =
    INTRINSIC_DIM, centers=3, cluster_std=1.2)

if INPUT_DIM > 2: # embedding two-dimensional data in higher
    dimensions
    v1 = np.random.uniform(low=-1, high=1, size=INPUT_DIM)
    v2 = np.random.uniform(low=-1, high=1, size=INPUT_DIM)
    while np.all(v1 == 0):
        v1 = np.random.uniform(low=-1, high=1, size=INPUT_DIM)
    while np.all(v2 == 0):

```

```

        v1 = np.random.uniform(low=-1, high=1, size=INPUT_DIM)
        # Creating two random vectors in  $\mathbb{R}^n$  and ensuring that they are
        # not zero vectors

        u1 = v1 / np.linalg.norm(v1)
        # normalizing v1

        ortho = v2 - np.dot(v2,u1) * u1
        # Gram-Schmidt orthogonalization for the second vector

        u2 = ortho / np.linalg.norm(ortho)
        # normalizing ortho

        # Now u1 and u2 are two orthonormal vectors in  $\mathbb{R}^n$  and we
        # create
        # a matrix with u1 and u2 as the rows
        embed_matrix = np.vstack((u1,u2))

        X_2d = copy.copy(X)
        # storing the original 2-dimensional data points

        X = np.matmul(X,embed_matrix)
        # creates the new data points

data = torch.from_numpy(X).float().to(DEVICE)

model = NF(INPUT_DIM, HIDDEN_DIM, N_LAYERS).to(DEVICE)
optimizer = optim.Adam(model.parameters(), lr=LR)

print("The dimension of the input is ",INPUT_DIM)
print("Starting training...")
model.train()

# just an example how it can be structured
losses = []
condition_numbers_mean = []
condition_numbers_max = []
condition_bound_mean = []

corr_coeff_first_layer = []
corr_coeff_second_layer = []
corr_coeff_sec_to_last_layer = []
corr_coeff_last_layer = []

# for a triangular matrix we use the lower bound
# abs(max diagonal element) / abs(min diagonal element)

for epoch in range(N_EPOCHS + 1):
    permutation = torch.randperm(data.size(0))
    for i in range(0, data.size(0), BATCH_SIZE):
        indices = permutation[i:i+BATCH_SIZE]
        batch_x = data[indices]

        optimizer.zero_grad()
        jac = model.compute_jacobians(batch_x)

```

```

# dummy to compute and log condition numbers
cn_over_batch = []
cn_bound_over_batch = []

for i in range(len(jac)):
    # iterating over the layers
    for j in range(len(batch_x)):
        # iterating over the batch in the layer
        matrix = jac[i][j].detach().cpu()
        U, S, V = torch.linalg.svd(matrix)
        cn = S.max() / S.min()
        #cn = np.max(jac[i][j].detach().cpu().numpy()) #
dummy
        cn_over_batch.append(cn)

        abs_diag = torch.abs(torch.diagonal(matrix))
        cn_bound = torch.max(abs_diag) / torch.min(abs_diag)
    )
        cn_bound_over_batch.append(cn_bound)

z, log_det_J = model(batch_x)
log_prob_z = -0.5 * torch.sum(z**2, dim=1) - 0.5 *
INPUT_DIM * np.log(2 * np.pi)
log_prob_x = log_prob_z + log_det_J

if losses != []:
    previous_loss = loss_item
else:
    previous_loss_torch = -torch.mean(log_prob_x)
    previous_loss = previous_loss_torch.item()
    # for the first batch; this makes the previous_loss
equal the same as the loss;
    # therefore their ratio will be 1

loss = -torch.mean(log_prob_x)
loss_item = loss.item()

losses.append(loss_item)

arr_cn_over_batch = np.reshape(np.array(cn_over_batch), (len
(jac), len(batch_x)))
arr_log_prob_cn = np.insert(arr_cn_over_batch, 0, log_prob_x.
tolist(), axis=0)
corrcoef_matrix = np.corrcoef(arr_log_prob_cn)
corrcoef_batch_list = corrcoef_matrix[0]

corr_coeff_first_layer.append(corrcoef_batch_list[1])
corr_coeff_second_layer.append(corrcoef_batch_list[2])
corr_coeff_sec_to_last_layer.append(corrcoef_batch_list
[-2])
corr_coeff_last_layer.append(corrcoef_batch_list[-1])

condition_numbers_mean.append(np.nanmean(cn_over_batch))
condition_numbers_max.append(np.nanmax(cn_over_batch))

```

```

        condition_bound_mean.append(np.nanmean(cn_bound_over_batch)
    )

    loss.backward()
    optimizer.step()

    if epoch % 50 == 0:
        print(f"Epoch {epoch}/{N_EPOCHS}, Loss: {loss.item():.4f}")

model.eval()
with torch.no_grad():
    z_samples = base_distribution.sample((N_SAMPLES,))
    x_generated = model.inverse(z_samples).cpu().numpy()
    z_transformed, _ = model(data)
    z_transformed = z_transformed.cpu().numpy()

if INPUT_DIM > 2:
    X = X_2d # 2-dimensional data points generated at the
             beginning

    transp_embed_matrix = np.transpose(embed_matrix)
    # transposed embedding matrix

    x_generated = np.matmul(x_generated, transp_embed_matrix)
    # mapping the subspace in  $R^n$  back to  $R^2$ 

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle('NF for dataset X', fontsize=16)

axes[0].set_title("Target Distribution")
axes[0].scatter(X[:, 0], X[:, 1], alpha=0.6, c='blue')
x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
axes[0].set_xlim(x_min, x_max)
axes[0].set_ylim(y_min, y_max)
axes[0].set_aspect('equal', adjustable='box')

axes[1].set_title("Samples from Base  $\rightarrow$  Target")
axes[1].scatter(x_generated[:, 0], x_generated[:, 1], alpha=0.6, c=
'green')
x_min, x_max = x_generated[:, 0].min() - 0.5, x_generated[:, 0].max
() + 0.5
y_min, y_max = x_generated[:, 1].min() - 0.5, x_generated[:, 1].max
() + 0.5
axes[1].set_xlim(x_min, x_max)
axes[1].set_ylim(y_min, y_max)
axes[1].set_aspect('equal', adjustable='box')

axes[2].set_title("Data  $\rightarrow$  Base")
axes[2].scatter(z_transformed[:, 0], z_transformed[:, 1], alpha
=0.6, c='red')
x_min, x_max = z_transformed[:, 0].min() - 0.5, z_transformed[:,
0].max() + 0.5
y_min, y_max = z_transformed[:, 1].min() - 0.5, z_transformed[:,
1].max() + 0.5
axes[2].set_xlim(x_min, x_max)

```

```

axes[2].set_ylim(y_min, y_max)
axes[2].set_aspect('equal', adjustable='box')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])

plt.figure(figsize=(12, 6))
plt.subplot(2, 2, 1)
plt.plot(losses)
plt.title('Loss over Batches')
plt.xlabel('Batch')
plt.ylabel('Loss')

plt.subplot(2, 2, 2)
plt.plot(condition_numbers_mean)
plt.title('Mean Condition Number over Batches')
plt.xlabel('Batch')
plt.ylabel('Condition Number')

plt.subplot(2, 2, 3)
plt.plot(condition_numbers_max)
plt.title('Maximum Condition Number over Batches')
plt.xlabel('Batch')
plt.ylabel('Condition Number')

plt.subplot(2, 2, 4)
plt.plot(condition_bound_mean)
plt.title('Mean Condition Number Bound over Batches')
plt.xlabel('Batch')
plt.ylabel('Condition Number Bound')

plt.tight_layout()

plt.figure(figsize=(12, 6))
plt.subplot(2, 2, 1)
plt.plot(corr_coeff_first_layer)
mean_y = np.mean(corr_coeff_first_layer)
plt.axhline(mean_y, color='red', linestyle='--', linewidth=2, label
            =f'Mean = {mean_y:.2f}')
plt.title('Correlation log_prob_x and cond. number of first layer')
plt.xlabel('Batch')
plt.ylabel('Correlation coefficient')

plt.subplot(2, 2, 2)
plt.plot(corr_coeff_second_layer)
mean_y = np.mean(corr_coeff_second_layer)
plt.axhline(mean_y, color='red', linestyle='--', linewidth=2, label
            =f'Mean = {mean_y:.2f}')
plt.title('Correlation log_prob_x and cond. number of second layer')
plt.xlabel('Batch')
plt.ylabel('Correlation coefficient')

plt.subplot(2, 2, 3)
plt.plot(corr_coeff_sec_to_last_layer)
mean_y = np.mean(corr_coeff_sec_to_last_layer)
plt.axhline(mean_y, color='red', linestyle='--', linewidth=2, label
            =f'Mean = {mean_y:.2f}')

```

```

    =f'Mean = {mean_y:.2f}')
plt.title('Correlation log_prob_x and cond. number of second to
    last layer')
plt.xlabel('Batch')
plt.ylabel('Correlation coefficient')

plt.subplot(2, 2, 4)
plt.plot(corr_coeff_last_layer)
mean_y = np.mean(corr_coeff_last_layer)
plt.axhline(mean_y, color='red', linestyle='--', linewidth=2, label
    =f'Mean = {mean_y:.2f}')
plt.title('Correlation log_prob_x and cond. number of last layer')
plt.xlabel('Batch')
plt.ylabel('Correlation coefficient')

plt.tight_layout()
plt.show()

```